Architecture

Package Layout

```
Fleet_Management/
  pyproject.toml             Python >= 3.10; extra "dashboard"
  README.md
  spec/                      this specification: spec.tex, references.bib
  examples/                  example inputs and regression scripts
  tests/                     pytest suite
  experiments/               experiments: inputs, runners, job scripts
  results/validator_tests/   fixtures of the validator tests (test data)
  euler/                     two Slurm jobs of the ETH Euler cluster
  src/fleet_management/
    __init__.py              public API
    config.py                reader, checks, broadcasting; load_config
    solver.py                solve(): checks, dispatch, result files
    horizon_sweep.py         sweep_operating_horizons, sweep_horizon_grid
    greedy_warm_start.py     build_greedy_warm_start
    validator_dashboard.py   Streamlit entry point of the dashboards
    degradation_model/
      base.py                shared model, registry, objective,
                             warm starts, result extraction,
                             performance counters
      gaussian.py, inverse_gaussian.py, wiener.py, gamma.py, rainflow.py
                             cell builders; gamma.py computes the cap A_1
      gamma_utils/
        gamma_replay_validator.py      replay validator
        gamma_stochastic_validator.py  Monte Carlo validator
        gamma_tail_validator.py        exact validator
        gamma_tail_bound.py            moschopoulos_tail_probability
    utils/plotter.py         plot_management, plot_horizon_sweep
    validation/
      validator.py           validate
      dashboard_inspector.py, dashboard_gamma.py
```

`examples/regression/` holds the scripts of the two Slurm jobs (Slurm is the job scheduler of the cluster): the job `gamma_horizon_convergence.sbatch` runs `run_horizon_sweep.py`, and the job `vbz_progressive_feasibility.sbatch` runs `run_progressive_vbz_feasibility.py`.

Data Flow of One Solve

```
solve(input_path, results_path="output.yaml", *,
      warm_start=None, warm_start_mode="start") -> dict
```

1. Check the input file: it exists (`FileNotFoundError`) and has the extension `.yaml`, `.yml`, `.json`, `.h5` or `.hdf5` (`ValueError`). Check `results_path` (Section **??**).
2. Read the file with the input reader (Section **??**) into a mapping.
3. `load_config`: normalise the mapping into a `FleetConfig` and run the checks of Section **??** (`ValueError`).
4. Build the model (Section 1.4), apply the warm start (Section 3) and optimise.
5. Extract the result (Section **??**): the shared extraction, then the `extract` hook of each builder. Write the result file (Section **??**) and return the result.

Steps 3 to 5 form the internal entry point `solve_mapping(data, results_path=None, *, warm_start=None, warm_start_mode="start")`, which takes the mapping of step 2. Gurobi errors are not caught.

Public API

The package exports:

- one solve: `solve` (Section 1.2);
- horizon sweeps: `sweep_operating_horizons` and `sweep_horizon_grid` (Section 2);
- plots: `plot_management` and `plot_horizon_sweep` (Section 4);
- the structural validator `validate` (Section 5.1);
- the Gamma validators (Section 5.3): the replay validator `validate_gamma_replay_schedule` with its file version `validate_gamma_replay_files`; the Monte Carlo validator `validate_gamma_stochasti` and the selector of both, `validate_gamma_schedule`, with its file version `validate_gamma_schedule_files`

Internal functions, not exported: `solve_mapping` and `load_config` (Section 1.2); the greedy heuristic `build_greedy_warm_start` (Section 3.6); the exact Gamma validator `validate_gamma_tail_bound_sch` and its file version (Section 5.3).

Cell-Builder Interface

A registry maps each model name (`gaussian`, `inverse_gaussian`, `wiener`, `gamma`, `rainflow`) to a builder; the model of cell (i, ℓ) is its entry of `model`.

```
register_cell_builder(name, builder)  # CELL_BUILDERS[name] = builder
get_cell_builder(name)    # NotImplementedError, with the names, if
    absent
dispatch_cell(ctx, i, l)  # get_cell_builder(ctx.model_of[i, l])
                          #     .add_cell(ctx, i, l)
```

A builder has the attribute `name` and three hooks:

- `prepare(ctx, cfg, cells, opts)`: called once per model in the fleet, with its cells. It stores the per-cell arrays of the model in `ctx`, creates the variables of the tracked quantities, anchors and selectors of these cells only, may tighten variable bounds, and sets Gurobi's `NonConvex` to 2 if some cell keeps an exact quadratic row.
- `add_cell(ctx, i, l)`: adds the rows of one cell (Section **??**): dynamics in the form of `encoding`, replacement and anchor rows, repair-cost rows, reliability rows in the form of `reliability_impl`, loop rows of the full tracked state.
- `extract(ctx, cfg, out)`, optional: adds model-specific entries to the result `out` after the shared entries.

The context `ctx` holds the sizes F, L, M, H_1, H_2, T; the Gurobi model; the shared variables x, x^m, x^r (absent if replacement is not allowed), μ, z (cells with $C_R > 0$) and u (if $C_D > 0$), with the shapes of Section **??**; the per-cell arrays (model, τ, ε, ρ, initial and replacement values, starting anchors); the increment accessor $(i, j, \ell, k) \mapsto$ increment after the phase lookup, Eq. (**??**), with $j = 0$ the depot; the run options; a dictionary `extras`.

Building order:

1. Create the model, apply the run options, create the shared variables.
2. Call `prepare` for each model present, in alphabetical order of the names.
3. Build the objective (Section **??**) and the rows that do not depend on the model: assignment (Section **??**), depot day (Section **??**), damage regularisation (Section **??**).
4. Call `add_cell` for every cell, i outer, ℓ inner.

Each builder defines $\mu_{i\ell k}$, $k = 1, \ldots, T$, for its cells. A model is added by a module with its builder, its registration, its input checks in `config.py` (Section **??**) and its section in this specification.

Horizon Sweeps

A horizon sweep solves one input for several pairs (H_1, H_2) and ranks the cases. `sweep_operating_horizons` varies H_2 for one H_1; `sweep_horizon_grid` varies H_1 and H_2. No input key starts a sweep, and `solve()` never runs one.

Cost Measures and Ranking

J_{init} and J_{op} of a case are Eq. (**??**) on its solution. The cost per operating day, and the projected cost over an evaluation horizon $E > H_1$, are

$$\bar{J}_{\text{op}} = J_{\text{op}}/H_2, \qquad P_E = J_{\text{init}} + (E - H_1)\,\bar{J}_{\text{op}}. \tag{1}$$

A case is *feasible* if its result holds a solution and its status is not `"infeasible"`, `"unbounded"` or `"inf_or_unbounded"`. A case is *proven* if it is feasible and its status is `"optimal"` at the MIP gap used for the case: Gurobi's `MIPGap` after all options are set, `gurobi_params` included (Section **??**); it is the entry `requested_mip_gap` of `performance`. The *ranking value* is $\bar{J}_{\text{op}}$ in `sweep_operating_horizons`; in `sweep_horizon_grid` it is P_E if an evaluation horizon is given, otherwise $\bar{J}_{\text{op}}$. The *best proven case* and the *best feasible case* have the smallest ranking value among the proven and the feasible cases. A tie goes to the smaller H_2; in the grid, to the smaller H_1, then the smaller H_2.

`sweep_operating_horizons`

```
sweep_operating_horizons(
    input_path, operating_horizons, *, output_path=None,
    stop_on_gradient=False, gradient_tolerance=1e-3,
    flat_gradients_required=2, minimum_cases=3,
    maximum_mip_gap_for_stopping=None,
    warm_start=False, warm_start_mode="start", n_workers=1,
    objective_mode="operating_average", evaluation_horizon=None) -> dict
```

Arguments with a rule:

- `input_path`: H_1 is the first entry of `H`, or `H` if it is a scalar; the second entry is ignored.
- `operating_horizons`: the values of H_2; positive integers, at least one, no duplicates; evaluated in increasing order.
- `output_path`: the report file, `.yaml` or `.yml`; missing parent folders are created; `None` writes no file.
- The arguments of the early stop (Section 2.3): `gradient_tolerance` is finite and ≥ 0; `flat_gradients_required` is ≥ 1; `minimum_cases` is ≥ 2; `maximum_mip_gap_for_stopping` is `None` or a finite number ≥ 0.
- `objective_mode`, `evaluation_horizon`: written into every case input; `"evaluation_total"` needs $E > H_1$. For `warm_start`, `warm_start_mode` and `n_workers`, see Sections 3.5 and 2.5.

Steps:

1. Read the input file with the input reader (Section **??**); the data must be a mapping (`TypeError`). Check every argument.
2. For each H_2, the case input is the input data with `H` $= [H_1, H_2]$, and with `objective_mode` and `evaluation_horizon` (if given) replaced. Time-indexed data are not interpolated: a profile whose length is not H_2 fails the checks of Section **??**, as an error of that case.
3. Solve the case input with `solve_mapping` (Section 1.2) and `results_path=None`; no temporary file is written. Keep the row of the case, not its schedule.
4. After each case: append its row, annotate the gradient (Section 2.3), test the stop rule, write a checkpoint if `output_path` is given. At the end, write the final report and return it.

Errors. `ValueError` before the first case for an invalid argument, for a list `H` whose length is not two, and for `warm_start` or `stop_on_gradient` with `n_workers` > 1; such an error is never recorded as a case error. An error inside a case is captured in its row (Section 2.4).

Early Stop

Let $\bar{J}_n$ be the cost per operating day of the n-th evaluated case. If cases $n-1$ and n are feasible, the relative gradient is

$$\hat{g}_n = \frac{\bar{J}_n - \bar{J}_{n-1}}{(H_{2,n} - H_{2,n-1}) \max\{\, |\bar{J}_{n-1}|,\ \varepsilon_{\text{mach}} \}}, \tag{2}$$

with $\varepsilon_{\text{mach}}$ the machine precision of double-precision numbers. With $t =$ `gradient_tolerance`, the *class* of case n is `"increase"` if $\hat{g}_n > t$, `"decrease"` if $\hat{g}_n < -t$, `"flat"` otherwise, and `"unavailable"` if $n = 1$ or one of the two cases is not feasible. A case *qualifies* if it reports a finite MIP gap at most its threshold: `maximum_mip_gap_for_stopping` if it is a number, otherwise the MIP gap used for the case (`requested_mip_gap` of its row). The gradient of case n is *eligible to stop* if cases $n-1$ and n qualify. Every row carries its gradient, class and eligibility, also if `stop_on_gradient` is false. With `stop_on_gradient=True` the sweep stops after case n if $n \geq$ `minimum_cases`, the gradient of case n is eligible, and either the class of case n is `"increase"` (reason `"cost_increase"`) or the last `flat_gradients_required` cases up to n all have the class `"flat"` and an eligible gradient (reason `"flat_gradient"`). The last candidate of `operating_horizons` is a hard limit.

Report, Checkpoint and Errors

The report is a mapping of plain Python types, written as YAML. *Keys:* `input` (input path); `complete` (`false` in a checkpoint); `last_completed_H2`; `objective` (a text); `varied_parameter` (`"H2"`); `fixed_dimensions` (F, M, L) and `H1`; `planned_operating_horizons` and `evaluated_operating_horizo` `warm_start` (`enabled`, `mode`) and `n_workers`; `stopping_rule`, with `enabled`, `relative_gradient_tolerance_pe` the stop arguments `flat_gradients_required`, `minimum_cases` and `maximum_mip_gap_for_stopping` under their own names (the last one `null` for the default), the `reason` (`"in_progress"`, `"candidate_range_exhau` `"cost_increase"` or `"flat_gradient"`) and `stopped_early` (complete with fewer cases than planned); `cases` (the rows, by H_2); `formulation_growth` from the first to the last row with counts: `from_H2`, `to_H2`, `added_time_steps` and, for each count q of `formulation`, `added_`q and q`_per_added_time_step` (`null` with fewer than two such rows); for $b \in$ {`proven`, `feasible`}, `best_`b`_H2` and `best_`b`_J_op_average`, and `best_feasible_status` (`null` without such a case).

Keys of a case row: `H1`, `H2`, `T`, `dimensions` (F, M, L, H_1, H_2, T); from the result of the case (Section **??**), `status`, `objective_mode`, `evaluation_horizon`, `objective`, the scalar keys of the cost report, and the warm-start report `warm_start`; `objective_bound`, `mip_gap` and `absolute_mip_gap` $= \max\{0,$ objective$-$bound$\}$; `requested_mip_gap` and `requested_time_limit_seconds`, Gurobi's `MIPGap` and `TimeLimit` after all options are set; `optimizer_seconds`, `solutions_found`; `formulation`, the size entries of `performance` from `variables` to `nonzeros`; `timing`, the wall-clock entries of `performance` with `branch_and_bound_nodes`, `simplex_iterations` and `work_units`; the gradient keys `cost_gradient_per_H2`, `relative_cost_gradient_per_H2`, `gradient_classific` and `gradient_stopping_eligible` (Section 2.3).

Checkpoint. With `output_path`, the report is rewritten after every case, with `complete = false` and the reason `"in_progress"` until the sweep ends.

Case errors. An exception (class `Exception`) raised by a case after the arguments are accepted is logged at level ERROR with H_1 and H_2. The row of the case has the status `"error: "` followed by the message, and `null` in every entry except the sizes; the sweep continues. Such a row is neither feasible nor proven, and is never a warm-start source.

Parallel Cases

With `n_workers` > 1, up to `n_workers` cases are solved at the same time, each in its own process with its own Gurobi environment and one Gurobi licence token. The option is refused with `warm_start` and with `stop_on_gradient`. Rows arrive in the order of completion; the report orders them by H_2, and each rewrite recomputes the gradient annotations over the ordered rows.

`sweep_horizon_grid`

```
sweep_horizon_grid(
    input_path, transitory_horizons, operating_horizons, *,
    output_path=None, warm_start=True, evaluation_horizon=None,
    **sweep_options) -> dict
```

Both horizon lists hold positive integers; they are sorted and repeated values are removed. `evaluation_horizon`, if given, must exceed every H_1 (`ValueError`). `sweep_options` are the keyword arguments of `sweep_operating_horizons` except `objective_mode`. With `n_workers` $>$ 1, `warm_start=False` must be passed.

For each H_1 in increasing order, `sweep_operating_horizons` runs on the input data with that H_1 and the list of H_2, with `objective_mode` `"evaluation_total"` if an evaluation horizon is given and `"operating_average"` otherwise. Warm starts (Section 3.5), the early stop and the capture of case errors act within each H_1; the early stop uses $\bar{J}_{\text{op}}$ also when E is given. The grid report is rewritten after every case (`complete = false`) and at the end.

Keys: `input`, `complete` and `objective` as in Section 2.4; `comparison_metric`, the ranking key (`"projected_evaluation_cost"` or `"J_op_average"`); `mip_certificate_metric` (`"objective"`); `evaluation_horizon`; `varied_parameters` (`["H1", "H2"]`); `fixed_dimensions` (F, M, L); the candidates `planned_transitory_horizons` and `planned_operating_horizons`; the flag `warm_start_enabled`; `H1_sweeps`, the report of each operating sweep, with the input file of the grid as `input`; `cases`, all rows by H_1, then H_2; `best_proven` and `best_feasible`, `null` or a mapping with `H1`, `H2`, `T`, `status`, `mip_gap`, `objective` and the scalar keys of the cost report except `J_op` and `J_total`.

Warm Starts

A warm start gives Gurobi values for the binary variables. Four mechanisms exist: a *start* (attribute `Start`) from a source, with an optional completion; *hints* (attribute `VarHintVal`) from a source; a *relaxation start*; and the *greedy heuristic*, which builds a source from the input.

Interface and Reports

`solve()` takes `warm_start`, a source or `None` (default), and `warm_start_mode`: `"start"` (default), `"hint"` or `"both"`; another value raises `ValueError`. A *source* is a result dictionary or the dictionary returned by the greedy heuristic (Section 3.6). It carries `x` (shape $F' \times (M'+1) \times T'$) and the integers `H1`, `H2` with $T' = H_1' + H_2'$; it may carry `m` and `r` (shape $F' \times L' \times T'$). A source whose `x` is missing or `null` has no effect. F', M', L' may differ from F, M, L: values are set for the indices that both have. The result keys `warm_start` and `relaxation_warm_start` report what each mechanism did (Section **??**).

Day Mapping

Day $k \in \{1, \ldots, T\}$ takes the values of the source day

$$\sigma(k) = \begin{cases} k, & k \le \min\{H_1, H_1'\}, \\ H_1' + 1 + \big(\max\{0,\ k - H_1 - 1\} \bmod H_2'\big), & \text{otherwise.} \end{cases} \tag{3}$$

The mapped values, rounded to the nearest integer, are $x_{ijk} \leftarrow x'_{ij\sigma(k)}$ for $j = 0, \ldots, M$; $x^m_{i\ell k} \leftarrow m'_{i\ell\sigma(k)}$, or 0 if the source has no `m`; and $x^r_{i\ell k} \leftarrow r'_{i\ell\sigma(k)}$ if `allow_replacement` is true and the source has `r`. A binary variable that these values fix through its defining rows (the selector $n_{i\ell k}$ of Eq. (**??**)) gets that value.

Start, Hints and Completion

Mode `"start"` sets `Start` of every mapped variable; continuous variables get no value. Mode `"hint"` sets `VarHintVal` instead, with no `Start`, no completion and no `VarHintPri`. Mode `"both"` sets `Start` and `VarHintVal`. In the modes `"start"` and `"both"` the completion runs if the source requests it.

Completion. Only the source requests it, with `complete_binary_start = true`; the greedy heuristic does. It runs if the mapping set at least one value, with the time limit `completion_time_limit` of the source (seconds, default 60).

1. Copy the model; fix every binary variable that has a `Start` value to that value; the other binary variables stay free. Optimise the copy within the time limit.
2. If the copy has a solution, set `Start` of every variable of the original model to its value in the copy.
3. Otherwise record the status of the copy. If it is infeasible, compute an irreducible infeasible subset (IIS, Gurobi `computeIIS`) of the copy and record the names of at most 50 of its linear rows and general constraints.

The original model is not changed; a failed completion does not stop the solve.

Relaxation Start

If the run option `relaxation_warm_start` (Section **??**) is true and no start or hint was applied (`applied` is false in the report `warm_start`):

1. Copy the model and relax its linear rows with Gurobi's `feasRelaxS` (sum of absolute violations); integrality, variable bounds and general constraints are kept.
2. Optimise the copy with the time limit max{1, `relaxation_time_limit`} seconds and the MIP gap 0.05.
3. If the copy has a solution, set `Start` of every integer variable of the original model to the rounded value of the variable of the same name in the copy.

The relaxed solution is not a point of the original model.

Warm Starts in Sweeps

With `warm_start=True`, the source of a case is the result of the nearest preceding case of the same H_1 that has a solution, used in the mode `warm_start_mode`. The first case of each H_1 has no source; a case with an error or without a solution is never a source.

Greedy Heuristic

```
build_greedy_warm_start(cfg, *, repair_trigger=0.70)
    -> (dict, GreedyWarmStartReport)
```

Module `fleet_management.greedy_warm_start`, not exported; it works for every model. `cfg` is the normalised configuration (`load_config`, Section 1.2); $r =$ `repair_trigger` must lie in $[0, 1]$ (`ValueError`). The returned source has the keys `H1`, `H2`, `x`, `m`, `r`, `complete_binary_start` (`true`) and `completion_time_limit` (60).

Construction pass. A proxy state $\hat{s}_{i\ell}$ starts at $\mu_{0,i\ell}$. For each day $k = 1, \ldots, T$:

1. Order the missions by decreasing $\max_{i,\ell} \mu^{\text{inc}}_{i\ell jk}/\tau_{i\ell}$; a tie goes to the larger j.
2. In this order, give each mission to the train i without a mission on day k that minimises, in lexicographic order, $\big(\max_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell},\ \sum_\ell(\hat{s}_{i\ell} + \mu^{\text{inc}}_{i\ell jk})/\tau_{i\ell}\big)$; a tie goes to the smaller i. Add $\mu^{\text{inc}}_{i\ell jk}$ to $\hat{s}_{i\ell}$. The check $F > M$ (Section **??**) guarantees a train for every mission.
3. A train without a mission is idle, unless some component has $\hat{s}_{i\ell} > r\,\tau_{i\ell}$: then $x_{i0k} = 1$, and every such component is repaired ($x^m_{i\ell k} = 1$, $\hat{s}_{i\ell} \leftarrow (1 - \rho_{i\ell})\,\hat{s}_{i\ell}$ for every repair law).

The construction pass schedules no replacement.

Loop pass. For each cell, compute the reference trajectory ζ (Section **??**) of the current schedule from the initial values, starting anchors and replacement values of the inputs. The excess e of the cell is the largest of the loop excesses $\max\{0,\ \zeta^q_T - \zeta^q_{H_1}\}/M_q$ of every tracked quantity and anchor q with natural range M_q (Section **??**), and of the violations of the reliability row on every day, divided by the natural range of the bounded quantity. The row is taken in the form built at the cell (Sections **??**–**??**): the exact row for `"exact"`, the tangent row for `"tangent"`, and for `"pwl"` the cap of the segment that contains the mean of the day (Section **??**). While $e > 10^{-10}$: for every free day $k \in \{H_1 + 1, \ldots, T\}$ (the train has no mission, the cell no intervention), put a repair of the cell on day k (the train goes to the depot) and recompute e; keep the repair with the smallest e, a tie to the later day. If no repair lowers e by more than 10^{-10} and replacement is allowed, repeat the trial with a replacement. If no trial lowers e, count one failure for the cell and go to the next cell. A start with failures is returned.

Report. `GreedyWarmStartReport.as_dict()` returns plain Python types. Keys: `repairs` and `replacements`, the numbers of ones in `m` and `r`; `repeatability_feasible`, true without a failure; `repeatability_failures`, the number of cells with a failure; `maximum_repeatability_excess`, the largest raw excess over all cells after the loop pass; `maximum_proxy_ratio`, the largest $\hat{s}_{i\ell}/\tau_{i\ell}$ of the construction pass and reference mean over $\tau_{i\ell}$ of the loop pass; `assignment_order`, for each day, the missions in the order treated.

Plotting API and Layout

Both plots are in `utils/plotter.py`.

Schedule Plot

```
plot_management(input_file_path, plot_file_path=None) -> None
```

`input_file_path` is a result file of `solve()` (YAML, JSON or HDF5). `plot_file_path` is a `.png` or `.pdf` image, default `"output.png"`; a path without extension gets `.png`. Errors: `FileNotFoundError` for a missing result file or image folder; `PermissionError` for an image folder that is not writable; `ValueError` for another extension of either file. Any result with a solution is plotted, whatever its status. `ValueError`, naming the keys, if the result lacks one of `F`, `M`, `L`, `H1`, `H2`, `tau`, `mu_0`, `mu`, `x`; if a shape differs from Section **??**; if $H_1 + H_2 \neq T$; or if

`component_names` has a length other than L. Missing `m` and `r` are read as zero; the default name of component ℓ is "Component ℓ" (ℓ from 1); the model of each row is read from the key `model`.

Layout. A grid on the left, a summary panel on the right.

- Rows: one per cell (i, ℓ), ordered by train, then component. Each label has two lines, "Train i — *component name*" and the model name. A heavier line separates the trains.
- Columns: "initial" ($\mu_{0,i\ell}$) and the days $1, \ldots, T$; a vertical dashed line after day H_1; the labels "transitory" and "operating" above the grid.
- Colour: a green-to-red scale of $\mu_{i\ell k}/\tau_{i\ell}$, clipped to $[0, 1]$; colour bar "tracked mean / failure threshold".
- Marks: one per field of a day, given by the first rule of the table that applies; none in the column "initial". A value counts as 1 if it is above 0.5. A legend names the five marks.

Mark	Rule for cell (i, ℓ) on day k
`P`	replacement: $x^r_{i\ell k} = 1$
`R`	repair: $x^m_{i\ell k} = 1$
`M`$_j$	train i flies mission $j \geq 1$: $x_{ijk} = 1$ (the mark shows j)
`D`	depot day without intervention: $x_{i0k} = 1$, $x^m_{i\ell k} = x^r_{i\ell k} = 0$
`I`	idle day of the train (Section **??**)

The summary panel "Run summary" shows the values that the result holds: status (coloured by outcome), objective, sizes and horizon, models and repair laws present, objective mode with J_{op}/H_2, MIP gap in percent, numbers of variables (all, continuous, integer) and linear rows, optimizer time, branch-and-bound nodes, and the numbers of repairs and replacements. The counts come from `performance`; if a Gurobi log `runner.out` lies in the folder of the result file, the panel shows instead the presolved counts of its lines "Presolved: r rows, c columns" and "Variable types: a continuous, b integer", and says so.

Sweep Plot

```
plot_horizon_sweep(sweep_report_path, plot_file_path=None) -> None
```

`sweep_report_path` is one path, or a list of paths, of reports of `sweep_operating_horizons` (YAML or JSON); `ValueError` for another file type or an empty list, `FileNotFoundError` for a missing file. `plot_file_path`: as for `plot_management`, with the default `"horizon_sweep.png"`. The reports must agree in F, M, L, H_1, and each needs a list `cases` (`ValueError`). For equal H_2 the plot keeps the case with a solution, then the smaller J_{op}/H_2, then the larger objective bound. The title names the sweep and F, M, L, H_1. Three panels over H_2 show: (a) J_{op}/H_2 of each case, with one mark for a proven case (status `"optimal"`), one for a feasible case that is not proven, and one for a case without a feasible solution (error rows included), a mark on the best feasible case, and the values of the proven cases; (b) the relative MIP gap in percent, with a horizontal line at `maximum_mip_gap_for_stopping` of `stopping_rule` if it is a number, otherwise a mark at the requested MIP gap of each case; (c) the numbers of continuous variables, integer variables and linear constraints.

Validation Tools

The tools check a saved result against its input; they do not call the optimiser. Each tool reads the input with the input reader and `load_config` (Section 1.2), reads a result file in any format that `solve()` writes, and takes the model of each cell from the result key `model`.

Structural Validator

```
validate(input_path, results_path, validation_path="validation.yaml",
         tol=1e-6) -> dict
```

The report is written to `validation_path` (`.yaml`, `.yml` or `.json`; no extension gets `.yaml`; the folder is created) and returned. Errors: `FileNotFoundError` for a missing file; `PermissionError` for a folder that is not writable; `ValueError` for an unsupported extension, a result without `x` (missing or `null`), an input that fails Section **??**, or sizes F, M, L, H_1, H_2 or array shapes that differ between result and input (named in the message). The status is not a check: a feasible incumbent is validated like an optimal solution, and the report echoes the status. A check passes if its violation is at most `tol`. The checks, each with its violation:

- `binary`: x, x^m, x^r take only the values 0 and 1; the largest distance of an entry from $\{0, 1\}$.
- `one_activity`: $\sum_{j=0}^{M} x_{ijk} \le 1$ (Section **??**); $\max\{0,\ \max_{i,k}(\sum_j x_{ijk} - 1)\}$.
- `mission_coverage`: every mission $j = 1, \dots, M$ has $\sum_i x_{ijk} = 1$ (the depot $j = 0$ is not summed); $\max_{j \ge 1, k} |\sum_i x_{ijk} - 1|$.
- `depot_gating`: $x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k}$ (Section **??**), and $x^r = 0$ if `allow_replacement` is false; the largest excess of the left side over the right side.
- `idle`, if the result has the key `idle` (Section **??**): $\max |\texttt{idle}_{i\ell k} - (x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k})|$.
- `loop_`q for $q \in \{\texttt{mu}, \texttt{v}, \texttt{R}, \texttt{K}, \texttt{g_mu}, \texttt{g_v}, \texttt{g_R}, \texttt{g_K}\}$: the loop row $q_{i\ell T} \le q_{i\ell H_1}$ (Section **??**), with `NaN` and `null` entries skipped; $\max\{0,\ \max_{i,\ell}(q_{i\ell T} - q_{i\ell H_1})\}$.
- `objective`: the saved objective o is not below the recomputed objective r; $\max\{0,\ r - o\} / \max\{1, |o|\}$. The recomputed objective is Eq. (**??**) in the objective mode of the input, with the day cost of Eq. (**??**) on the saved x, z (0 where `NaN`) and x^r, the costs of the input, and the saved u.

Keys: `status`, `solver_objective`, `recomputed_objective`, `tolerance`; `checks`, a list of records with `name`, `passed`, `violation` and `tolerance`; `passed`, true if all checks passed; `max_violation`.

Dashboards

Streamlit pages (extra `dashboard`), started with

```
python -m streamlit run src/fleet_management/validator_dashboard.py
```

The sidebar selects the mode, the input file, the result file and the tolerance (default 10^{-6}); a button loads the pair. The dashboards read only these two files. A failed load shows the error and no page.

Mode "Result inspector" (every model). Tabs: *Overview* (status, objective, models, H_1, H_2, T, number of reliability violations, largest μ/τ, cells with the smallest margin); *Reliability* (margins by train, component and day; filters for train, component and violations only); *Repeatability* (the loop row of each cell and saved tracked quantity, Section **??**); *Train timeline* (the means of one train with τ and the marks of Section 4); *Heatmap* (margins over cells and days); *Component comparison* (one component across the trains); *Validation* (the checks of `validate`); *Raw data* (with an export). The *margin* of a cell on a day is the right side minus the left side of the exact form of its reliability row (Sections **??**–**??**), on the saved tracked states. The row holds if the margin is at least $-$`tolerance`; `NaN` entries are left out.

Mode "Gamma diagnostic" (Gamma cells), which runs the replay and the exact validators (Section 5.3). Tabs: an overview (repairs, replacements, outcome of the replay validator); the tracked mean of each Gamma cell with its cap $A_1(\beta^*\tau, \varepsilon)/\beta^*$ (Section **??**); the exact tail probability of the true damage against ε, also as a heatmap; raw data. The mode needs no schedule inside the input file.

Validators of the Gamma Model

Each validator takes the normalised configuration `cfg` (`load_config`) and the result dictionary, and returns a report of plain Python types. Its file version takes an input path, a result path and `report_path` (default `None`); it writes the report as YAML or JSON (`ValueError` for another suffix) and creates the folder. `ValueError` if `cfg` has no Gamma cell; if a required array is missing, has a wrong shape or a non-finite entry; if an array that must be binary is not within `tolerance` of $\{0, 1\}$. `tolerance` (default 10^{-8}) must be positive. With `raise_on_failure=True`, a report with `valid = false` raises `AssertionError`.

Replay validator.

```
validate_gamma_replay_schedule(cfg, result, *, tolerance=1e-8,
                               raise_on_failure=False) -> dict
validate_gamma_replay_files(input_path, result_path, report_path=None,
                            **kwargs) -> dict
```

It reads `x`, `m`, `r` (optional), `mu`, `z` and, for ARD_1 cells, `g_mu`. Global violations (strings in `global_violations`): a result horizon other than T; a train with more than one activity; a mission not covered exactly once; a repair and a replacement of a cell on one day; an intervention on a day with a mission or without a depot day. For each Gamma cell and day k, the reference trajectory of μ and g^{μ} (Section **??**) is computed from the saved schedule, with the initial value, starting anchor and replacement value of the cell. The event of the day is `"replacement"`, `"repair"`, `"mission"` or `"idle"`. The errors are:

- `tracked_mean`: $\max\{0,$ reference mean $-$ $\mathtt{mu}_{i\ell k}\}$; `mean_anchor`: the same for `g_mu`, under ARD_1;
- `removed_mean`: $\max\{0,\ x^{m}_{i\ell k}\,\rho_{i\ell}(\mu_{i\ell,k-1} - g_{i\ell,k-1}) - z_{i\ell k}\}$, with $g = 0$ under ARD_{∞} (Section **??**); skipped where z is `NaN` ($C_{R,i\ell} = 0$);
- `cap`: $\max\{0,\ \mathtt{mu}_{i\ell k} - A_1(\beta^{*}\tau, \varepsilon)/\beta^{*}\}$ (Section **??**);
- `repeatability`: $\max\{0,\ \mathtt{mu}_{i\ell T} - \mathtt{mu}_{i\ell H_1}\}$, and the same for the saved anchor.

An error above `tolerance` adds a record `i`, `l`, `k`, `event`, `field`, `expected`, `saved`, `error` to `violations`. *Keys:* `validator`; `valid`, true without a global violation and without a violation; `solver_status`, `dimensions`, `gamma_cells`, `gamma_ard1_cells`, `transitions_checked`, `repairs`, `replacements`; `maximum_errors`, the largest error of each field; `global_violations`, `violations`, `timing`.

Exact validator (module `gamma_utils.gamma_tail_validator`, not exported).

```
validate_gamma_tail_bound_schedule(cfg, result, *, tolerance=1e-8,
    convolution_tolerance=1e-12, max_series_terms=100000,
    include_steps=False, raise_on_failure=False) -> dict
validate_gamma_tail_bound_files(input_path, result_path, report_path=
   None,
                                **kwargs) -> dict
```

For each Gamma cell and day $k = 1, \ldots, T$, the true damage is a list of independent Gamma terms (Section **??**). A mission adds a term with the exact rate of its increment. A repair multiplies the rate of the repairable terms by $1/(1-\rho)$, and removes them for $\rho = 1$; under ARD_1 the repaired terms join the frozen part of the anchor. A replacement starts a new list with the replacement term. An initial or replacement damage without a rate (`gamma_beta_0` or `gamma_beta_new` omitted) is a known constant c, not a term: the tail is $\Pr(\text{sum of the terms} > \tau - c)$, and 1 if $c > \tau$. The tail $\Pr(D_k > \tau)$ is computed by the Moschopoulos series (with equal rates, the Gamma tail); the upper bound is the partial sum plus the remaining mixture mass, which is at most `convolution_tolerance`. `RuntimeError` if the series does not reach this tolerance within

`max_series_terms` terms. `convolution_tolerance` must lie in $(0, 1)$; `max_series_terms` must be positive. A day passes if the upper bound is at most $\varepsilon_{i\ell} +$ `tolerance`, the saved mean is at least $\mathbb{E}D_k -$ `tolerance` and, under ARD_1, the saved anchor is at least the exact mean of the frozen part minus `tolerance`. *Keys:* those of the replay report; `minimum_reliability_slack`, the minimum of ε minus the upper bound; `worst_reliability`, the step that attains it; `maximum_remaining_mass`; the numbers of series terms and the timing of the tail computations; with `include_steps=True`, the list `steps` (terms and values of every cell and day).

Monte Carlo validator.

```
validate_gamma_stochastic_schedule(cfg, result, *, repetitions=100000,
    random_seed=20260908, maximum_schedule_failure_rate=None,
    confidence_level=0.95, batch_size=20000, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule(cfg, result, *, mode=None, repetitions=None,
    random_seed=None, maximum_schedule_failure_rate=None,
    confidence_level=None, batch_size=None, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule_files(input_path, result_path, report_path=None,
                              **kwargs) -> dict
```

The schedule is fixed, and only Gamma cells are sampled. In each of `repetitions` runs, the true damage of every Gamma cell is drawn: the initial damage, each mission increment and each replacement draw are independent Gamma variables with the means and rates of the inputs. Under ARD_1 the initial damage is the sum of two independent terms with means g_0^{μ} (the anchor) and $\mu_0 - g_0^{\mu}$; a damage without a rate is the constant of its mean. Each day the repair law acts on the sampled path: $D \leftarrow (1-\rho)\,D$ under ARD_∞; $D \leftarrow g + (1-\rho)(D-g)$, then $g \leftarrow D$, under ARD_1. A replacement draws the damage of the replacement part and, under ARD_1, sets g to it. After each day the run records whether $D > \tau$. A *schedule failure* is a run in which some Gamma cell exceeds its threshold on some day. The generator is seeded with `random_seed`, and draws are made in batches of `batch_size` runs; the same arguments give the same report. `confidence_level` must lie in $(0, 1)$; `repetitions` and `batch_size` must be positive; `maximum_schedule_failure_rate`, if given, must lie in $[0, 1]$. Each failure rate (of the fleet, of a cell, of the worst day of a cell) comes with its one-sided Clopper–Pearson upper bound at `confidence_level`: for f failures in n runs, the `confidence_level`-quantile of the law $\mathrm{Beta}(f+1,\ n-f)$, and 1 if $f = n$. `valid` is true if no target is given, or if the observed schedule failure rate is at most `maximum_schedule_failure_rate`. *Keys:* `validator`, `mode`, `valid`, `timing`, the arguments used; `scope` ("Gamma cells only"); `complete_fleet_validation` (false if the fleet has other cells); `excluded_cells` (`i`, `l`, `component`, `model`, `reason` of each other cell); `schedule` (`failed_replays`, `failure_rate`, `failure_rate_upper_confidence_bound`, `maximum_failure_rate`, `observed_target_met`, `confidence_qualified_target_met`); `cells`, for each Gamma cell `i`, `l`, `component`, `repair_model`, `threshold`, `analytical_epsilon`, `failed_replays`, `failure_rate` with its bound, `worst_time_step` with its failures, rate and bound, `maximum_sampled_damage`, `maximum_threshold_excess`.

`validate_gamma_schedule` runs the validators that `mode` selects: `"deterministic"` (replay), `"stochastic"` (Monte Carlo) or `"both"`, with the keys `deterministic` and `stochastic` and `valid` true if both are valid; another value raises `ValueError`. An argument that is `None` takes the value of the key of the same name in the input block `validation` (Section **??**), otherwise its default; the default of `mode` is `"deterministic"`.

References

References

[1] L. Doyen and O. Gaudoin. Classes of imperfect repair models based on reduction of failure intensity or virtual age. *Reliability Engineering & System Safety*, 84(1):45–56, 2004. doi:10.1016/S0951-8320(03)00173-X. Read in the authors' preprint submitted on 25 October 2002, whose page numbers are cited.

[2] S. Mercier and I. T. Castro. Stochastic comparisons of imperfect maintenance models for a gamma deteriorating system. *European Journal of Operational Research*, 273(1):237–248, 2019. doi:10.1016/j.ejor.2018.06.020.

[3] F. Roosta-Khorasani and G. J. Székely. Schur properties of convolutions of gamma random variables. *Metrika*, 78:997–1014, 2015. doi:10.1007/s00184-015-0537-9. Read in arXiv:1601.04731v1, whose page numbers are cited.

[4] G. J. Székely and N. K. Bakirov. Extremal probabilities for Gaussian quadratic forms. *Probability Theory and Related Fields*, 126:184–202, 2003. doi:10.1007/s00440-003-0262-6.